UNC741: PARALLEL & DISTRIBUTED COMPUTING

L	T	P	Cr
3	0	0	3.0

Course Objective: To understand the fundamentals of parallel and distributed programming and application development in different parallel programming environments.

Parallelism Fundamentals: Scope and issues of parallel and distributed computing, Parallelism, Goals of parallelism, Parallelism and concurrency, Multiple simultaneous computations, Programming Constructs for creating Parallelism, communication, and coordination. Programming errors not found in sequential programming like data races, higher level races, lack of liveness.

Parallel Architecture: Architecture of Parallel Computer, Communication Costs, parallel computer structure, architectural classification schemes, Multicore processors, Memory Issues: Shared vs. distributed, Symmetric multiprocessing (SMP), SIMD, vector processing, GPU, coprocessing, Flynn's Taxonomy, Instruction Level support for parallel programming, Multiprocessor caches and Cache Coherence, Non-Uniform Memory Access (NUMA).

Parallel Decomposition and Parallel Performance: Need for communication and coordination/synchronization, Scheduling and contention, Independence and partitioning, Task-Based Decomposition, Data Parallel Decomposition, Actors and Reactive Processes, Load balancing, Data Management, Impact of composing multiple concurrent components, Power usage and management. Sources of Overhead in Parallel Programs, Performance metrics for parallel algorithm implementations, Performance measurement, The Effect of Granularity on Performance Power Use and Management, Cost-Performance trade-off;

Distributed Computing: Introduction: Definition, Relation to parallel systems, synchronous vs asynchronous execution, design issues and challenges, A Model of Distributed Computations, A Model of distributed executions, Models of communication networks, Global state of distributed system, Models of process communication.

Communication and Coordination: Shared Memory, Consistency, Atomicity, Message- Passing, Consensus, Conditional Actions, Critical Paths, Scalability, cache coherence in multiprocessor systems, synchronization mechanism.

Parallel Algorithms design and Analysis: Parallel Algorithms, Parallel Graph Algorithms, Parallel Matrix Computations, Critical paths, work and span and relation to Amdahl's law, Speed-up and scalability, Naturally parallel algorithms, Parallel algorithmic patterns like divide and conquer, map and reduce, Specific algorithms like parallel Merge Sort, Parallel graph algorithms, parallel shortest path, parallel spanning tree, Producer-consumer and pipelined algorithms.

Course learning outcomes (CLOs): On completion of this course, the students will be able to

1. Identify the fundamentals of parallel and distributed computing including parallel architectures and paradigms.
2. Compare various parallel algorithms and key technologies.
3. Implement different parallel approaches for resolving real time problems like sorting, shortest path etc.
4. Analyze the communication and computation trade-offs in distributed systems.

Text Books:

1. C Lin, L Snyder. Principles of Parallel Programming. USA: Addison-Wesley (2008).
2. A Grama, A Gupra, G Karypis, V Kumar. Introduction to Parallel Computing, Addison Wesley (2003).

Reference Books:

1. B Gaster, L Howes, D Kaeli, P Mistry, and D Schaa. Heterogeneous Computing With Opencl. Morgan Kaufmann and Elsevier (2011).
2. T Mattson, B Sanders, B Massingill. Patterns for Parallel Programming. Addison-Wesley (2004).

3. Quinn, M. J.,Parallel Programming in C with MPI and OpenMP, McGraw-Hill (2004).

UNC*: HUMAN COMPUTER INTERFACE**

L	T	P	Cr
2	0	2	3.0

Course Objectives: This course aims to provide students with a solid foundation in human-computer interaction (HCI) and user-centered design principles. It emphasizes understanding human cognitive, perceptual, and physical aspects that impact interactive system design.

Foundations of Human Computer Interface: Introduction to HCI; importance of user-centered design; evolution of interaction paradigms; types of user interfaces including graphical, touch, voice, and conversational; overview of direct manipulation, mobile, wearable, AR/VR, and ambient systems.

Human Factors and Design Principles: Human capabilities and limitations affecting interface design—perception, memory, attention, and motor skills; cognitive models including Norman's interaction model, GOMS, KLM, Fitts' and Hick's laws; principles of usability and interaction design—Nielsen's heuristics, Shneiderman's rules, affordances, constraints, feedback, and Gestalt principles.

Interface Design Components and Visual Communication: Design of UI components—forms, buttons, menus, icons, dialogs, navigation structures; layout and alignment principles; typography and readability; visual hierarchy; color theory and contrast. Application of visual design in GUI for web, mobile, and embedded platforms.

Universal Design, Accessibility, and Evaluation: Principles of inclusive and universal design; accessibility standards (e.g., WCAG); assistive technologies; usability engineering lifecycle; rapid prototyping techniques; usability evaluation methods—heuristic evaluation, cognitive walkthroughs, think-aloud protocols, and A/B testing.

Information Architecture, Interaction and Web Interfaces: Structuring and organizing interface content; form design and validation; dialog structures and feedback handling; interaction styles—CLI, GUI, touch, gesture, voice; fundamentals of HTML, CSS, and responsive design; GUI design standards for web and mobile.

Intelligent Interfaces and Future Trends: Adaptive and intelligent interfaces—personalization, recommendation systems, conversational UI (chatbots, voice assistants); UX metrics and analytics—task success rate, error rate, SUS, NPS; emerging trends in HCI including emotion-aware computing, brain-computer interfaces (BCIs), XR environments, and ethical considerations.

Laboratory work: Introduction to interface evaluation and familiarization with design tools such as Figma or Adobe XD; wireframing and sketching of low-fidelity interfaces; interactive prototyping of basic UI components; application of typography, color theory, and mobile-first design principles; heuristic evaluation and redesign of existing user interfaces; accessibility and WCAG-compliant interface design; HTML/CSS-based UI development for computer science students or IoT/embedded interface design for electronics students; usability testing techniques including peer review and think-aloud protocols; design of voice or chatbot interfaces along with discussion on multimodal interaction; collection and reporting of UX metrics such as task success and error rate; a mini UX design sprint focused on solving a real-world interface